Safety and Guaranteed Stability through Embedded Energy-Aware Actuators

Gerrit A. Folkertsma, Stefan S. Groothuis, and Stefano Stramigioli

***Abstract*— Safety is essential for robots in unknown environments, especially when there is physical Human–Robot Interaction (pHRI). Control over energy, or passivity, is an effective safety mechanism. However, when the control algorithm is implemented in a discrete-time computer, computation and communication delays readily lead to loss of passivity and to instability. In this paper, a way to make the actuators aware of the energy that they inject into the system is presented. Passivity and stability are then always guaranteed, even in situations of total communication loss. These Embedded Energy-Aware Actuators are a model-free passivity and safety layer that make complex robotic systems dependable, well-behaved and safe. The proposed method is validated in simulation and experiments.**

I. Introduction

Robots have left the controlled environment of factories and are slowly coming into our lives. Apart from all cognitive challenges associated with this change, one of the major concerns is safety, especially when there is physical human–robot interaction (pHRI). There are a number of criteria defining "safety" in pHRI: amongst others the Head Injury Criterion, Maximum Power Index, and Maximum Mean Strain Injury Criterion. In general, to meet these criteria, and thus to be safe, the amount of power, energy, force, and/or acceleration transferred to the human must be limited [1]. Besides the Maximum Power Index which is directly related to energy, many of the other criteria, e.g., the strain and acceleration based measurements, ultimately depend on the energy transfer as well. Control strategies that limit this (mostly kinetic) energy present in the robot are successful: by estimating the energy in the robot, based on joint velocities, the position setpoint or maximum velocity are adapted to stay within safe energy levels [2], [3]. Besides these safety criteria and measures, the robot design paradigm has shifted from stiff, heavy, rigidly-controlled systems towards compliant, lightweight robots under impedance or interaction control [4]. Furthermore, the use of "soft actuators", i.e., series elastic or variable stiffness actuators that are back-drivable even under control failure, contributes to the intrinsic operational safety [5], [6]. More importantly, intrinsic safety may also be achieved by ensuring passivity of the robot. As proved in [7], if a system, e.g., the robot, is not strictly passive, it is always possible that a *passive* environment destabilises the system and extracts infinite energy from it, resulting in unstable behaviour. Therefore, robots operating in an unpredictable environment can be guaranteed to remain stable *if and only if they are passive*.

This work was financed by the Netherlands Organisation for Scientific Research with project no. 022.002.025, and the European Unions Horizon 2020 programme under grant no. 688857.

The authors are affiliated with the Robotics and Mechatronics group, CTIT institute, University of Twente. Email: {g.a.folkertsma,s.stramigioli}@ieee.org, s.s.groothuis@utwente.nl.

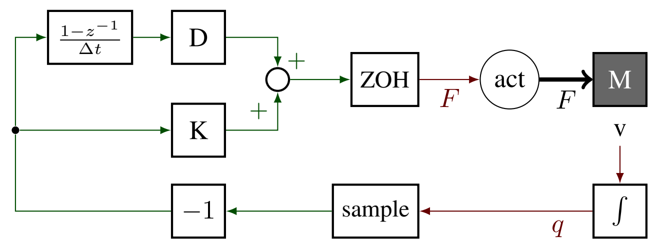


Fig. 1: Discrete-time implementation of a PD-controller. The sample block samples the position at each sample time; the ZOH (Zero Order Hold) block holds the previous force command during a sample timestep. The continuous-time implementation does not have these blocks, and has a normal derivative for the D-action in place of the finite difference block.

Recently, a passivity-based control scheme for pHRI was presented in [8]. A task-based energy-tank method was introduced, which cancels out non-passive terms in the system by properly choosing the tank dynamics. At the same time, the tank allows the system to use a certain maximum amount of energy to execute a particular task. Although these continuous-time control laws were successfully implemented on a robotic arm and were proved to be passive, it is important to consider the effects of time discretisation. Even a simple moving mass system with a virtual, albeit physically consistent, spring and damper implemented as a PD-controller, as shown in Figure 1, may lose its passivity due to the implementation of a controller and signal processing on a (digital) computer (see e.g. [9]). If, for instance, communication loss results in a failure of torque command updates to any of the low-level joint controllers, passivity cannot be guaranteed any longer. Time delays are a common problem in tele-manipulation and haptic interfaces, where passivity has already been used to stabilise systems subject to time delays. In [10], the Passive Set-Position Modulation (PSPM) method was proposed, which passivates a system by implementing a spring coupling with damping injection between a system's position and its commanded set-position. The Passivity Observer/Passivity Controller (PO/PC) as presented in [11], [12] implements a passivity observer, monitoring the energy flowing into and out of a system, while using a passivity controller to dissipate any excess energy, i.e., energy that was not first injected, that is generated by a system. Experiments were shown, for which a precise measurement

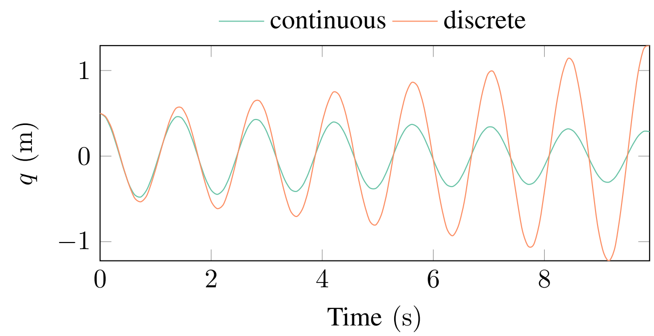


Fig. 2: Comparison between a passive continuous-time controller and the same controller in discrete-time, with a sampling frequency of 100 Hz. Though the controller acts as a passive spring and damper, the discrete-time implementation is unstable.

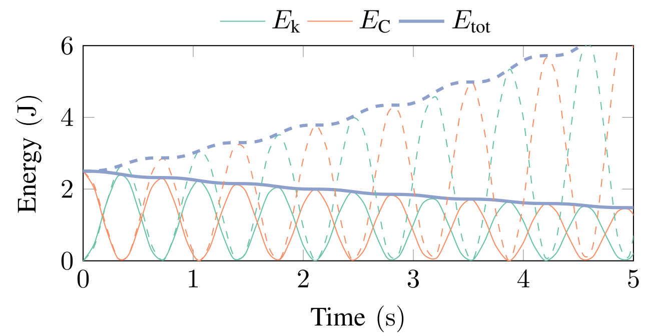


Fig. 3: The energy present in the system and controller: kinetic energy E_k in the mass and controller energy E_C in the virtual spring. Continuous line: continuous-time system; dashed line: discrete-time system.

of the interaction forces was necessary.

In this paper, a method is presented to add passivity to *any* electric-motor based actuator, *without* adding extra sensors like force/torque or position/velocity sensors. In contrast to previous work, the passivity layer is moved as close to the mechanical system as possible, right at the interface between the discrete time and continuous time system, i.e., at the actuators. The actuators supply energy to the system, which, due to a certain control law, may be unbounded. However, if the actuators are made passive, the whole system is always guaranteed to be passive [13]. Therefore, this paper presents the idea of Embedded Energy-Aware Actuators (E^2A^2): actuators that, on an embedded computing level, monitor and regulate the amount of energy they inject into the system. While safety depends on more than just energy, energy plays a large part in many safety criteria. By explicitly controlling energy and enforcing energy bounds, safety will be improved.

To illustrate the concept, the remainder of the paper starts with an elaboration on energetic properties in controlled systems and the problem of time discretisation in Section II. The concept of E^2A^2 is then presented in Section III, after which some case studies are described in Section IV. A discussion with future work ideas is given in Section V, and the paper concludes with Section VI.

II. Energy in Controlled Systems

A. System passivity

In physical systems, the property of passivity is an energy-based measure of stability. It is a special case of dissipativity, as introduced in [14], that arises naturally in physical dynamical systems. General dissipativity is defined by considering a system

$$\dot{x} = f(x, w), \quad z = g(x, w), \tag{1}$$

where x is the system state, w the input, and z the output, which take their values in their respective manifolds X, W, and Z. The supply function is a mapping

$$s : W \times Z \to \mathbb{R}. \tag{2}$$

The system (1) is said to be dissipative if there exists a storage function $V : X \to \mathbb{R}$ such that

$$V(x(t_1)) \ \leq \ V(x(t_0)) + \int_{t_0}^{t_1} s(w(t), z(t)) \, \mathrm{d}t \tag{3}$$

for $t_1 \geq t_0$. The system is conservative, i.e., non-dissipative, if the equality holds in (3). In physical systems, a natural choice for the storage function V is the total energy in the system; the supply function is the supplied power of the input port of the system, e.g., the duality product of mechanical force and velocity $\langle F, v \rangle$. Indeed, systems that contain only passive physical elements, i.e., masses, springs, and dampers, can never contain more energy than initially present. Consequently, the energy is bounded and thus a system that is overall passive is always stable [13], [15].

B. Passivity loss due to discretisation

The system in Figure 1 is an example of a physical system with only an inertia (1 kg), and an actuator. Because the PD-controller acts as a virtual spring ($20\,\mathrm{N\,m^{-1}}$) and damper ($0.1\,\mathrm{N\,s\,m^{-1}}$), the overall system is passive and, hence, stable—when implemented in continuous time. When this controller is implemented in discrete-time, as shown in Figure 1, sampling, computation delay, and a Zero Order Hold to keep controller commands constant during a sampling time, are added. The natural frequency of the system lies at 0.7 Hz, which means that a sampling frequency of 100 Hz is considerably higher than the system dynamics and should suffice. However, Figure 2 shows that simulating the behaviour of these systems in which the inertia has an initial position which preloads the virtual spring, results in a stable damped oscillatory behaviour in the continuous-time system, while the discrete-time system becomes unstable. The discrete-time system has lost its passivity by implementing the controller digitally, even using a rather high sampling frequency of 100 Hz relative to the system dynamics. Figure 3 shows the energies present in the system, i.e., kinetic energy of the inertia E_k, controller energy in the virtual spring E_C, and the total energy as their sum E_{tot}. The total energy decreases for the passive continuous-time system, but grows unboundedly for the non-passive discrete-time system.

Note that the original system was intentionally underdamped, and no special care was taken to do a proper discretisation, for instance by a stability-preserving bilinear transform. Furthermore, in practise a very high sample rate for the controller may be chosen, which mitigates the problem. However, it may not always be possible to obtain sufficiently high sample rates; and even so: it is always only mitigation, as there are no guarantees whatsoever on passivity. The point is that the E^{2}A^2-method is completely independent of the chosen controller or system, so the purpose here is to obtain a simple demonstration system for the E^{2}A^2-method. If instability due to time delays already occurs in this simple system, it is clear that additional measures have to be taken to ensure passivity, and thereby stability and safety, in any robotic or mechatronic system.

C. Energy sampling

A crucial step towards ensuring passivity of the controller is measuring how much energy it supplies to the system. It is possible to accurately estimate the energy supplied by the controller if two conditions are met:

1) the actuator force is constant over a sampling period;
2) a position sensor is collocated with the actuator.

Condition 2 is a straightforward design choice, while condition 1 is met if the actuator's inner control loop, e.g., the current controller of an electric motor, is sufficiently fast compared to the main control loop and a ZOH is used. When these conditions are met, the energy injected during a single sampling period T can be calculated as:

$$\begin{aligned} E_T &= \int_{t_i}^{t_{i+1}} P(t)\mathrm{d}t = \int_{t_i}^{t_{i+1}} \langle F(t), v(t)\rangle \,\mathrm{d}t \\ &= \int_{t_i}^{t_{i+1}} \langle F(t_i), \dot{q}(t)\rangle \,\mathrm{d}t = \langle F(t_i), q(t_{i+1}) - q(t_i)\rangle \,, \end{aligned} \tag{4}$$

with $P(t)$ the supplied power, $F(t)$ and $v(t) = \dot{q}(t)$ the motor force and velocity, respectively, and $q(t_i)$ the motor position at time step t_i. Writing the discrete-time equations in computable form, since (4) needs a motor position from a future time step, where $\bar{F}_{i-1}$ is the control force set at t_{i-1} and held constant until t_i by Condition 1, yields:

$$\hat{E}_i = \hat{E}_{i-1} + \bar{F}_{i-1} \cdot (q_i - q_{i-1}) = \hat{E}_{i-1} + \bar{F}_{i-1} \cdot \Delta q_i, \tag{5}$$

in which $\hat{E}_i$ is the energy estimate at time t_i.

The result of applying this energy sampling method to the system of Figure 1, is shown in Figure 4. Even at an even lower sampling frequency of 25 Hz, with a clearly unstable system, the energy is very accurately estimated.

This energy sampling method was introduced in [16] as "sampled passivity" for discrete Port-Hamiltonian Systems, with a focus on stable and passive tele-manipulation systems. The concept has until now only been used in the context of tele-manipulation chains, not as a tool to build passive architectures, other than in [17]. In this paper, this concept is used for the first time to make actuators intrinsically passive. In this new control paradigm, the actuator, as the interface between the continuous and discrete world, is responsible for ensuring passivity—not a high-level controller that is far from this interface.

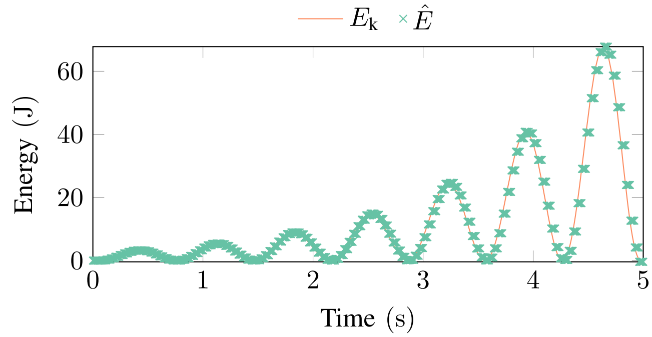


Fig. 4: Supplied energy estimated by "passive sampling", compared to the actual kinetic energy of the mass. To show the effectiveness, sampling frequency has been reduced to 25 Hz.

III. Embedded Energy-Aware Actuators

By embedding the energy sampling as explained in the previous section in an actuator, or as close to it as possible, the actuators keep track of the supplied or extracted energy themselves. These actuators are therefore called "Embedded Energy-Aware Actuators" (E^{2}A^2). To ensure passivity of the actuator, the available energy the actuator can use has to be limited. This gives rise to a finite energy budget, and necessitates the definition of a strategy to be applied when the energy budget has been depleted.

A. Energy budget

By allocating to the actuator an initial energy $E(t_0) = E_0$, (5) can be used to keep track of the "energy budget" E. Any energy the actuator supplies to the robot, i.e., positive work, must be taken from this budget, thereby lowering the available energy. Hence, when the budget has been depleted, i.e., $E = 0$, the actuator's power output must be zero for passivity to hold. Never more energy than what was initially present in the energy budget is then used, meaning that passivity is guaranteed. If the actuator extracts energy from the robot, i.e., negative work, this energy fills the budget, since injected energy $F_{i-1} \cdot \Delta q_i$ of (5) is negative.

B. Depleted energy strategy

The power output of the actuator, i.e. the injection of energy by the actuator into the mechanical part of the robot, can be forced to zero by setting either one of the dual power variables to zero: the applied force F, or the controlled velocity $\dot{q}$. It depends on the application which one is the more suitable strategy. Setting F to zero, for instance, still allows other (external) forces on the system to accelerate the robot and actuator. If a constant position is desired, $\dot{q}$ can be forced to zero.

The E^2A^2-type actuator has an embedded passivity layer between controller and actuator passivating the system by:

$$F_{\text{actuator}} = \begin{cases} F_{\text{controller}} & \hat{E} > 0 \\ \begin{cases} 0 & F_{\text{controller}} \cdot \hat{\dot{q}} \geq 0 \\ F_{\text{controller}} & F_{\text{controller}} \cdot \hat{\dot{q}} < 0 \end{cases} & \hat{E} \leq 0, \end{cases} \tag{6}$$

where F_{actuator} is the applied output force of the actuator, $F_{\text{controller}}$ is the desired controller force to be applied, and $\hat{\dot{q}}$ is the estimated actuator velocity.

C. Energy-included control messages

With the Embedded Energy-Aware Actuators being used, a control message communicated over a system bus is not any longer just a setpoint r for torque, velocity or position, but also includes the energy ΔE that may be expended for a task: $\sigma := (r,\ \Delta E)$. ΔE is added to the local energy budget E, while r is executed until the next setpoint σ is received, or until the energy budget has been depleted. Any varying time delay in the system bus will, therefore, not cause any stability problems.

With task-based energy budgets, a controlled amount of energy may be injected or "used" by the actuator. The initial budget allocation may, for instance, depend on (virtual) interaction energy change during impedance control, as was done in [8], expected potential energy change when lifting a robot arm, or friction losses.

IV. Case Studies

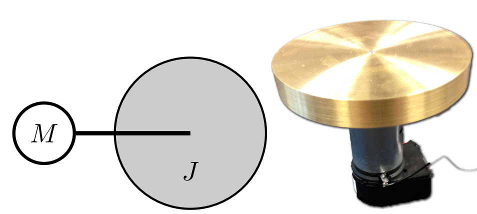
Two different experimental setups were constructed to show the working of the E^2A^2 in practice: the previously introduced example of an inertia with an actuator that implements a spring in Section IV-A, and a motor with a pulley that lifts a mass against gravity in Section IV-B. The diagrams and experimental setups are shown in Figure 5. The former setup shows instability in oscillations similar to the previous simulations, which is solved by the new actuator. The latter demonstrates the accuracy of the energy estimate, which should be approximately equal to the mass's potential energy, proportional to its height.

The electric motors of both setups are driven by a motor controller[1] operating in current control mode, which also runs the energy budget algorithm. A computer, communicating with the motor controller over an RS232 serial connection, sets the torque/current command r and allocates the available energy ΔE, such that $\sigma := (r,\ \Delta E)$.

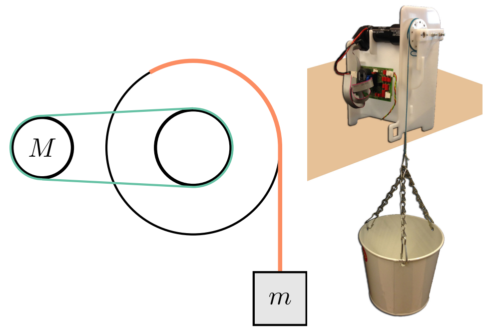
Furthermore, simulations with a 3-DoF robot arm with and without the use of Embedded Energy-Aware Actuators were done in Section IV-C to show the applicability to more complex systems.

A. Virtual spring

The very simple experimental setup illustrated in Figure 5a consists of an electric motor[2] with a heavy brass disk bolted directly to the motor shaft. The control algorithm



(a) A motor M that acts as a virtual rotational spring for a large inertia J.



(b) A motor M that lifts a mass m by means of a pulley.

Fig. 5: Systems used for testing the E^2A^2 architecture.

on the computer implements a spring with a stiffness of $3.98\,\text{mN}\,\text{m}\,\text{rad}^{-1}$, which results in an eigenfrequency of $0.51\,\text{Hz}$ with the disk's inertia of $3.76 \times 10^{-4}\,\text{kg}\,\text{m}^2$. The algorithm runs at a sampling frequency of $50\,\text{Hz}$.

1) Regular actuator: First, the experiment is carried out without the embedded passivity layer: the controller is started and the disk is given a small initial push to start oscillation. Figure 6 shows that the disk quickly spins out of control and the motor keeps injecting energy into the system. Clearly, also a very simple system will quickly become unstable, even though the sampling frequency is two orders of magnitude higher than the eigenfrequency.

2) E^2A^2-actuator: With the E^2A^2-concept implemented, the initial energy budget of $E = 0.1\,\text{J}$ is included in the first control message, while the low-level controller keeps track of the injected energy. Now, the passive controller is indeed passive: whenever the budget has been depleted, i.e., $E = 0$, the current to the motor is set to $0\,\text{A}$, which corresponds to $F_{\text{actuator}} = 0$, according to (6). This prevents further energy injection, but still allows energy extraction, e.g., when braking the inertia, to refill the energy budget and continue the desired task. Hence, the disk keeps oscillating.

One could argue that the stability of this system is determined by the controller settings, e.g., P(ID) gains. Although a stable system might be obtained by changing the gains, which means changing the virtual stiffness, the behaviour will change which may be undesired. Moreover, stability is only obtained within margins. The E^2A^2-implementation is not dependent on controller gains and corresponding stability margins, since the energy that is injected into the system, which directly relates to its behaviour and stability, is monitored and controlled. Likewise, increasing the sampling frequency in an ad hoc way from $50\,\text{Hz}$ to $1\,\text{kHz}$ may stabilise this particular system by decreasing computation delays, it

[1]Elmo SimplIQ Solo Whistle.
[2]maxon A-max 26, 110201.

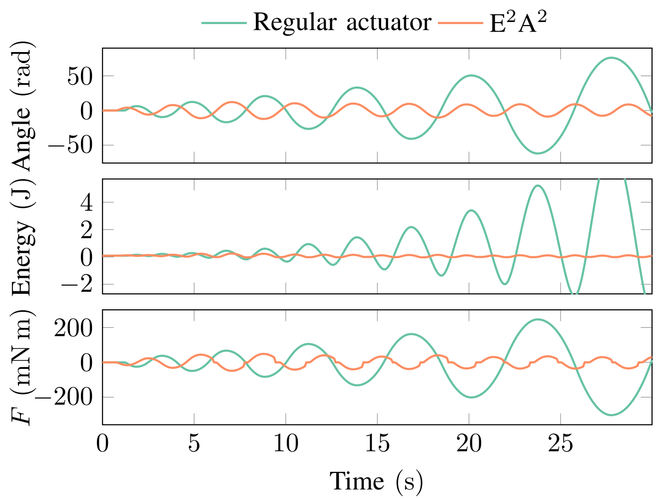


Fig. 6: Experimental results for the virtual spring setup depicted in Figure 5a. Note that the torque is frequently cut off to passivate the system when the energy budget is depleted, resulting in a stable system with bounded energy.

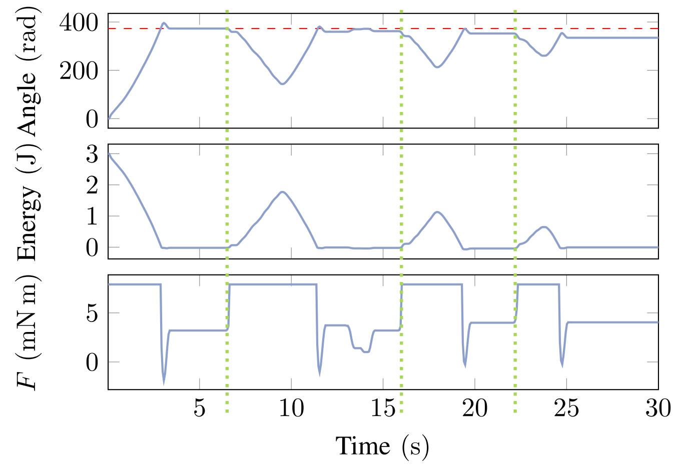


Fig. 7: Lift experiment results. The mass is pulled downwards at 6.5 s, 16 s and 22.5 s (which injects energy into the actuator), and is lifted at $t = 12\,\mathrm{s}$ (which does nothing: the cable slacks). The red, dashed line shows that a little energy is lost to friction every time the mass moves down and up. Between $t = 22\,\mathrm{s}$ and 28 s, the computer was disconnected.

is no guarantee that that is sufficient for another system. The E^2A^2-implementation is independent from sampling frequencies, since it regulates injected energy directly.

B. Lift

The lift experiment consists of a motor with gearbox[3] and a pulley, which lift a mass, as shown in Figure 5b. The control command is a constant torque r that lifts the mass off the ground, which requires an amount of energy equal to the potential energy gained by the mass. The initial energy allocation is $E = 3\,\mathrm{J}$. When this budget is depleted, the motor velocity will be controlled to 0 by the E^2A^2. Note that instead of a constant torque, any torque or even velocity profile can be applied; the E^2A^2-method works independently from the high-level control law and the actual system.

Figure 7 shows the experimental results. Indeed, the mass is halted shortly before the motor has rotated 400 rad, because the energy budget has been depleted. With a gearbox ratio of $33 : 1$ and pulley radius of 2 cm, this corresponds to a height of 23 cm. The mass weighs 816 g, so the potential energy gain is only about 1.8 J. Apparently, a lot of the controller energy of 3 J is lost to friction. In order to verify this, we have modelled the system with the modelling and simulation program 20-sim [18]. The mechanical friction is implemented as a Coulomb-viscous friction model according to

$$\tau_R = \tau_{\text{Coulomb}} \cdot \operatorname{sgn}(\omega) + R_{\text{viscous}} \cdot \omega. \tag{7}$$

After fitting the friction parameters based on the measured velocity and position ($\tau_{\text{Coulomb}} = 3.0 \times 10^{-5}\,\mathrm{N\,m}$; $R_{\text{viscous}} = 4.0 \times 10^{-5}\,\mathrm{Nm/(rad/s)}$), the simulation confirms that 1.2 J is lost in motor/gearbox friction.

With the energy loss estimate, extra energy could be added to the budget and achieve the desired height of 35 cm, albeit at the risk of overshooting if the actual friction is less than the modelled friction. First, the simulation was used to estimate the energy lost to friction when the mass is pulled all the way up to 35 cm, which turned out to be 1.8 J. When this energy was added to the budget at the start of the experiment, on top of the original 3 J, the mass does indeed reach the intended height of 35 cm, confirming that an initial energy budget may be dependent on friction losses, as previously mentioned.

The experiment shown in Figure 7 can be viewed as a video attachment to this paper, during which a communication loss was caused by unplugging the RS232 serial cable. It can be seen that the system continues to operate normally, which shows that an Embedded Energy-Aware Actuator makes a system completely robust to (varying) communication delays.

C. Robotic arm

A simulation of a 3-DoF robotic arm with and without the use of Embedded Energy-Aware Actuators was performed, to demonstrate the application of the E^2A^2-method for energy safety guarantees on a more complicated system. Although this is a computer simulation, the previous experiments have shown that the E^2A^2-approach works in practice, guaranteeing passivity and precise control of the amount of injected energy. The arm consists of three links and three rotational joints (Figure 8). In the reference configuration ($q_1 = q_2 = q_3 = 0$) the joint axes are z, y, and x, respectively. Each link is 0.5 m long, and weighs 1 kg, uniformly distributed over a link. Each joint is modelled as a torque actuator with a viscous friction of $2.0\,\mathrm{N\,m\,s\,rad^{-1}}$.

The controller implements an impedance control on the end-effector, with a stiffness of $K = 20\,\mathrm{N\,m^{-1}}$, and has active gravity compensation according to (9). τ is the torque sent to the joint actuators and τ_{gravity} is the calculated joint torque due to gravity. The Jacobian J of (8) is derived from the robot

[3] maxon DC motor 118746, maxon gear 166938.

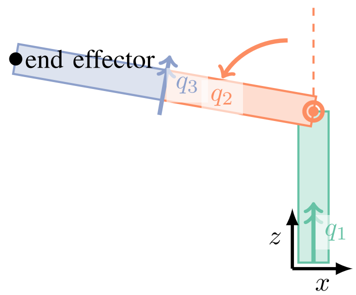


Fig. 8: The robotic arm consists of three links with three rotational joints. Each link is $0.5\,\mathrm{m}$ long and weighs $1\,\mathrm{kg}$. When $q_i = 0, \forall i$, the arm points straight up.

kinematics. $\boldsymbol{x}$ is the $\begin{pmatrix} x & y & z \end{pmatrix}^\top$ position of the end-effector or setpoint; $\boldsymbol{q}$ are the joint positions $\begin{pmatrix} q_1 & q_2 & q_3 \end{pmatrix}^\top$.

$$\boldsymbol{v}_{\text{ee}} = J(\boldsymbol{q}) \cdot \dot{\boldsymbol{q}} \tag{8}$$

$$\boldsymbol{\tau}_{\text{impedance}} = J(\boldsymbol{q})^\top \cdot K \left(\boldsymbol{x}_{\text{setpoint}} - \boldsymbol{x}_{\text{ee}}\right) \tag{9}$$

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{impedance}} - \boldsymbol{\tau}_{\text{gravity}} \tag{10}$$

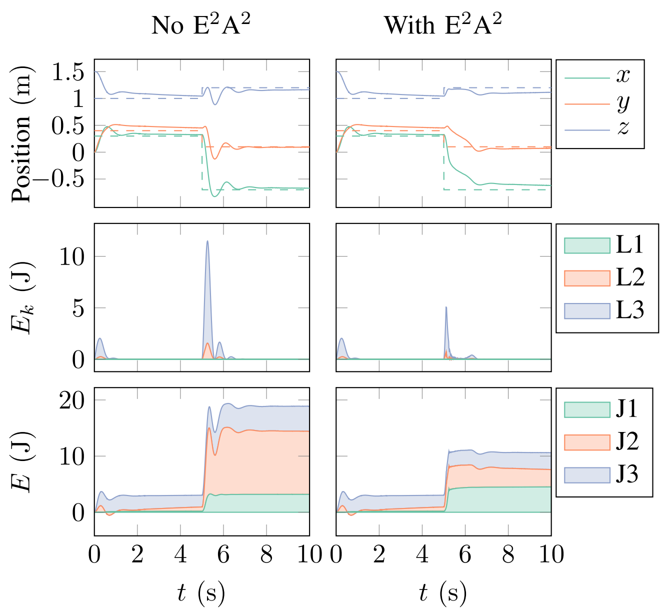


Fig. 9: Robot arm simulation experiment without (left) and with (right) E^2A^2. From top to bottom: end-effector setpoint and position, kinetic energy of links (Li), and energy injected by the joints (Ji). During a large setpoint change, the kinetic energy reaches high levels. Note that the energy plots are stacked.

The control law of (10) is implemented in discrete-time, with a frequency of $100\,\mathrm{Hz}$, and simulated 12-bits angular encoders on each joint. The actuators are implemented as Embedded Energy-Aware Actuators from the lift experiment. At the start of the simulation, the arm starts in its zero configuration (straight up). At $t = 0\,\mathrm{s}$ the setpoint is $\boldsymbol{x} = \begin{pmatrix} 0.3 & 0.4 & 1.0 \end{pmatrix}^\top \mathrm{m}$, and at $t = 5\,\mathrm{s}$ it changes to $\boldsymbol{x} = \begin{pmatrix} -0.7 & 0.1 & 1.2 \end{pmatrix}^\top \mathrm{m}$. The simulation is carried out twice: once without the energy-limit, during which the torque from (10) is sent to the motors directly, and once with the E^2A^2-scheme in place at each actuator. Because there is a significant amount of potential energy change from the first to the second setpoint, each actuator is given an energy budget of $4\,\mathrm{J}$ in the E^2A^2-simulation.

Figure 9 shows the results of the simulation without the E^2A^2-scheme on the left. The end-effector is pulled towards the setpoint by the impedance controller. Due to the large distance between the first and second setpoint, the kinetic energy reaches levels of $12\,\mathrm{J}$ during that change. In total, the actuators inject almost $20\,\mathrm{J}$.

In the second simulation, shown on the right of Figure 9, the E^2A^2 limits the total amount of energy injected, which also keeps the maximum kinetic energy bounded. Therefore, the velocity of the arm is decreased and the settling time of the arm reaching the setpoint is longer than before. This is a direct and expected effect of the desired lower kinetic energy, and, therefore, safer operation.

V. Discussion

The work presented here is an initial step towards the realisation of Embedded Energy-Aware Actuators. Future work will focus on a more sophisticated strategy in case the energy budget has been depleted. Setting either the actuator force or velocity to zero will prevent further energy injection into the system, it may sometimes be too conservative for some tasks. Adding additional limited energy for a limited number of times still results in a limited (cumulative) energy budget, which may be allowed in particular situations. Furthermore, and in line with this, the initial energy allocation to an actuator for a task is quite important and not yet treated in detail here. The focus of this paper is the model-free method of energy control; the allocation of energy budgets makes use of this method and *will* depend on the robot model and specific task. Closely linked to this is the question of performance, which we also did not address thoroughly in this paper: E^2A^2 is *only* a (model-free, guaranteed) passivity layer between controller and system. Within the constraints of a limited energy budget—which may limit the performance directly by constraining e.g. the maximum velocity—it is still possible to optimise for performance criteria. Also, quantisation effects of signals and general measurement uncertainty are important in being able to ensure passivity.

VI. Conclusion

Robots in unknown environments benefit from precise control over the amount of energy present in the physical system for safety reasons, especially in case of physical Human–Robot Interaction. Therefore, a method has been presented that adds model-free energy-awareness and passivity to any electric-motor based actuator, without the use of additional sensors, i.e., Embedded Energy-Aware Actuators (E^2A^2). This is realised by the definition of an energy budget that may be used by the actuator to perform a task, by monitoring the amount of energy injected from this budget into the system by the actuator, and by taking appropriate measures to avoid injecting additional energy when the budget has been depleted during the execution of a task. By a novel implementation of this passivity layer locally at the actuator, embedded in the low-level controller, overall system passivity and stability, and therefore safety, are guaranteed. This approach is completely robust to communication and computation delays, and works even under communication loss. Experiments have shown that implementing the E^2A^2-concept indeed ensures passivity and stability, and a simulation with a 3-DoF robotic arm has demonstrated the effect of applying the E^2A^2-method on a more complicated system. These case studies confirm that the E^2A^2-method is a promising approach to implement safety through passivity.

REFERENCES

[1] R. Alami et al., "Safe and dependable physical human-robot interaction in anthropic domains: State of the art and challenges," in *Intelligent Robots and Systems, 2006 IEEE/RSJ International Conference on*, Oct 2006, pp. 1–16.

[2] T. Tadele, T. de Vries, and S. Stramigioli, "The safety of domestic robotics: A survey of various safety-related publications," *Robotics Automation Magazine, IEEE*, vol. 21, no. 3, pp. 134–142, Sept 2014.

[3] M. Laffranchi, N. Tsagarakis, and D. Caldwell, "Safe human robot interaction via energy regulation control," in *Intelligent Robots and Systems, 2009. IROS 2009. IEEE/RSJ International Conference on*, Oct 2009, pp. 35–41.

[4] S. Haddadin, S. Parusel, R. Belder, and A. Albu-Schäffer, "It is (almost) all about human safety: A novel paradigm for robot design, control, and planning," in *Computer Safety, Reliability, and Security*, ser. Lecture Notes in Computer Science, F. Bitsch, J. Guiochet, and M. Kaâniche, Eds. Springer Berlin Heidelberg, 2013, vol. 8153, pp. 202–215.

[5] N. Tsagarakis, M. Laffranchi, B. Vanderborght, and D. Caldwell, "A compact soft actuator unit for small scale human friendly robots," in *Robotics and Automation, 2009. ICRA '09. IEEE International Conference on*, May 2009, pp. 4356–4362.

[6] A. Albu-Schäffer et al., "Anthropomorphic soft robotics – from torque control to variable intrinsic compliance," in *Robotics Research*, ser. Springer Tracts in Advanced Robotics, C. Pradalier, R. Siegwart, and G. Hirzinger, Eds. Springer Berlin Heidelberg, 2011, vol. 70, pp. 185–207.

[7] S. Stramigioli, "Energy-aware robotics," in *Mathematical Control Theory I: Nonlinear and Hybrid Control Systems*, ser. Lecture Notes in Control and Information Sciences, M. K. Camlibel, A. A. Julius, R. Pasumarthy, and J. M. Scherpen, Eds. Springer International Publishing, 2015, vol. 461, pp. 37–50.

[8] C. Schindlbeck and S. Haddadin, "Unified passivity-based cartesian force/impedance control for rigid and flexible joint robots via task-energy tanks," in *IEEE International Conference on Robotics and Automation*, May 2015, pp. 440–447.

[9] X. Xu, N. Ozay, and V. Gupta, "Passivity degradation in discrete control implementations: An approximate bisimulation approach," in *54th IEEE Conference on Decision and Control*, Dec 2015, pp. 6817–6822.

[10] D. Lee and K. Huang, "Passive-set-position-modulation framework for interactive robotic systems," *Robotics, IEEE Transactions on*, vol. 26, no. 2, pp. 354–369, April 2010.

[11] J.-H. Ryu, D.-S. Kwon, and B. Hannaford, "Stable teleoperation with time-domain passivity control," *Robotics and Automation, IEEE Transactions on*, vol. 20, no. 2, pp. 365–373, April 2004.

[12] J.-H. Ryu, C. Preusche, B. Hannaford, and G. Hirzinger, "Time domain passivity control with reference energy following," *IEEE Transactions on Control Systems Technology*, vol. 13, no. 5, pp. 737–742, September 2005.

[13] A. v. d. Schaft, *L2-gain and passivity in nonlinear control*. Springer-Verlag New York, Inc., 1999.

[14] J. C. Willems, "Dissipative dynamical systems part i: General theory," *Archive for rational mechanics and analysis*, vol. 45, no. 5, pp. 321–351, 1972.

[15] R. Ortega et al., "Control by Interconnection and Standard Passivity-Based Control of Port-Hamiltonian Systems," *IEEE Trans. Automat. Contr.*, vol. 53, no. 11, pp. 2527–2542, 2008.

[16] S. Stramigioli, C. Secchi, A. Van der Schaft, and C. Fantuzzi, "Sampled data systems passivity and discrete port-hamiltonian systems," *Robotics, IEEE Transactions on*, vol. 21, no. 4, pp. 574–587, Aug 2005.

[17] Y. Brodskiy, "Robust autonomy for interactive robots," Ph.D. dissertation, University of Twente, Enschede, February 2014.

[18] Controllab Products B.V., "20-sim," August 2016. [Online]. Available: http://www.20sim.com/